

CS-E4650 Methods of Data Mining, Homework N

Clustering tendency and entropy-based measures

Student Name (student id 000000) and Student Name (student id 000000)

September 27, 2026

1 Methods

All computations were made in Python 3.11 with NumPy 2.4 (loading and checking the data, discretisation, entropies, greedy searches and simulations) and Matplotlib 3.11 (figures); the report was written in $\text{\LaTeX}$ with the course template. The script in the Appendix reproduces every figure, table and number of this report. We first verified that the data file contains 1000 three-dimensional vectors whose values all lie in $[0, 1]$ (the smallest value is exactly 0 and the largest exactly 1; 7 values lie on the boundary). For every non-empty subset of k dimensions, the cube $[0, 1]^k$ was divided into $m = 64$ equal cells by splitting each of its dimensions into $64^{1/k}$ equal ranges, the fraction of points in each cell was counted, and the probability-based entropy of equation (6.2) of Aggarwal's book [1] was computed (Section 4). The greedy backward and forward searches were simulated on the seven resulting entropy values. Beyond the required experiments we made four small extra experiments, reported in Section 9: (i) the plain Shannon entropy instead of equation (6.2), (ii) a Monte Carlo baseline of the entropy of uniformly distributed data (10 000 simulated data sets), (iii) the correlation and mutual information of each pair of components with a permutation test of independence, and (iv) all entropies and both greedy searches with $m = 729$ cells.

2 Histograms

Figure 1 shows the histograms of the three components with 64 equal-width bins on $[0, 1]$; these bins are the same 64 ranges that are used for the one-dimensional entropies. The dashed line marks the count $1000/64 \approx 15.6$ that is expected in every bin if a component is uniformly distributed.

Component 0 is unimodal but strongly skewed: a pronounced peak at about 0.2–0.3 is followed by a broad shoulder that extends to about 0.9, and both tails are thin. The distribution is clearly not uniform, and the combination of a narrow peak with a wide plateau hints at several overlapping groups, but there is no empty region that would separate them. On its own, component 0 therefore shows only a weak clustering tendency: the points cannot be split into well-separated groups along this axis.

Component 1 is flat. The bin counts fluctuate between 6 and 25 around the uniform level without systematic peaks or gaps, which is what random sampling from a uniform distribution produces. Component 1 shows no clustering tendency at all.

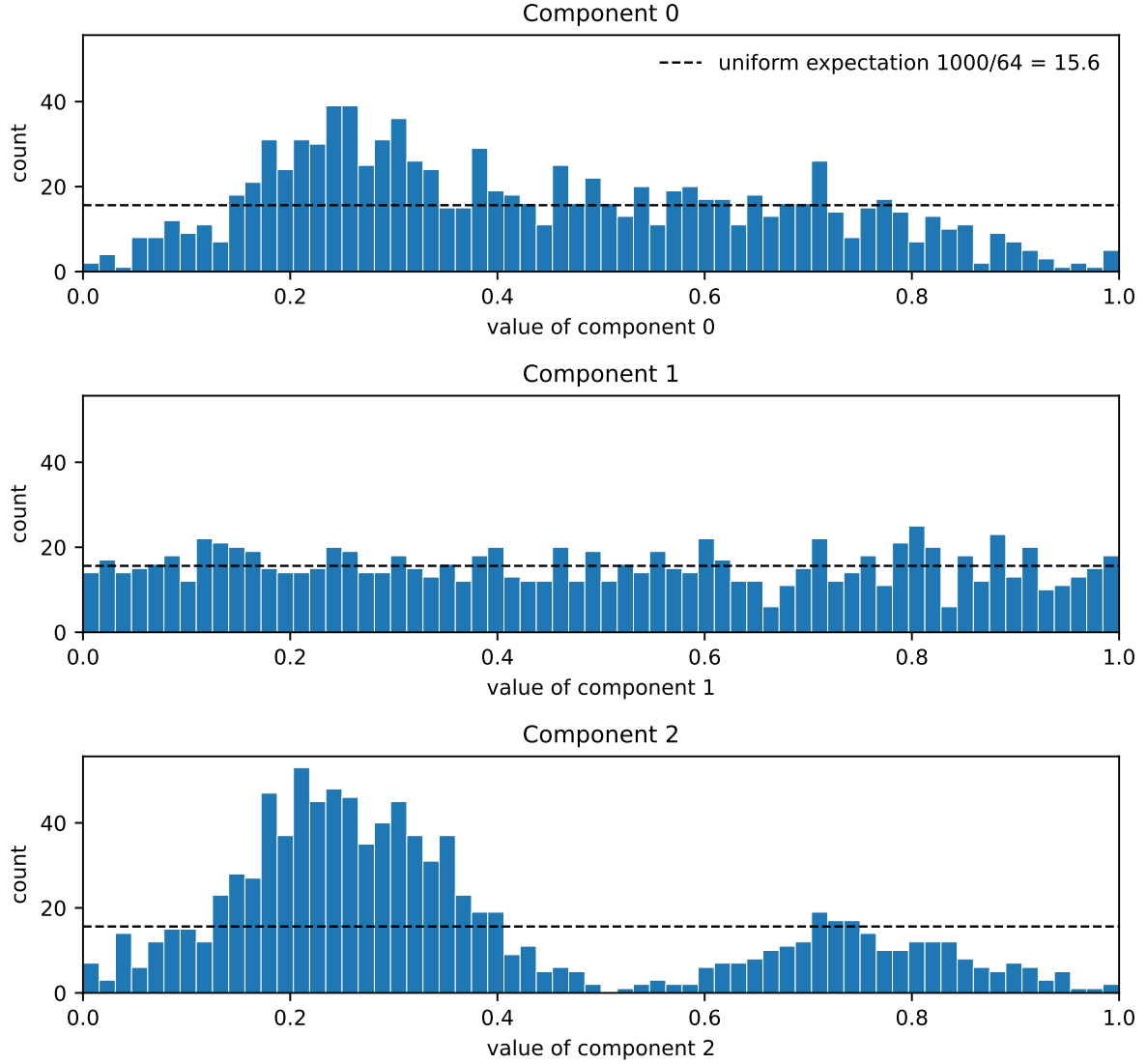


Figure 1: Histograms (64 bins) of component 0 (top), component 1 (middle) and component 2 (bottom). All panels share the same count axis. Dashed line: expected count per bin for a uniform distribution.

Component 2 is clearly bimodal. A large mode around 0.25 holds about three quarters of the points (762 values below 0.5), a smaller mode around 0.75 holds the rest (238), and between them there is an almost empty gap (only 18 values in $[0.45, 0.55]$). Component 2 alone already shows a strong clustering tendency with two natural groups.

Judged by these one-dimensional views alone, component 2 would contribute most to the clustering tendency, component 0 somewhat, and component 1 not at all.

3 Scatter plots

Figure 2 shows the scatter plots of the three pairs of components with equal scales on both axes. The light grey lines mark the 8×8 grid that is used for the two-dimensional entropies.

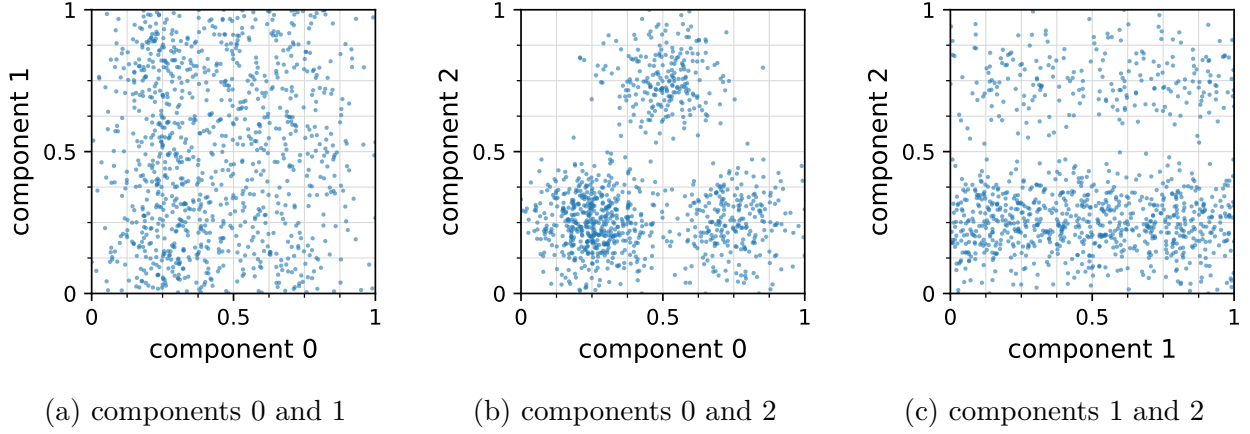


Figure 2: Scatter plots of the three pairs of components with equal aspect ratio; each axis is labelled with the component it shows. Grey lines: the 8×8 grid of the two-dimensional entropy calculation.

Components 0 and 1 (Figure 2a). The points fill the whole square without any groups. The density changes only along component 0 (a denser vertical band at 0.15–0.35 that thins out towards 1), and this pattern is the same at every value of component 1. The joint distribution thus looks like the product of the two marginal distributions: the components appear independent (Pearson correlation $r = 0.035$, and the mutual information is at the level of independent data, Section 9). Combining the uniform component 1 with component 0 only spreads the points out; it does not create clusters.

Components 0 and 2 (Figure 2b). Three compact, well-separated clusters appear: a large one around (0.25, 0.25) with about half of the points (511), and two smaller ones around (0.74, 0.25) (251 points) and (0.51, 0.75) (238 points).¹ The two components are strongly dependent: when component 2 is low, component 0 is bimodal (around 0.25 or 0.75), and when component 2 is high, component 0 is concentrated around 0.5. The dependence is not linear ($r = 0.152$ only), but the mutual information is about eight times the level of independent data (Section 9). As a consequence, neither one-dimensional projection shows all three clusters: in component 2 the two lower clusters coincide, and in component 0 the upper cluster falls between the two lower ones and fills the gap between them, which explains why the histogram of component 0 has no gap. Only the combination of components 0 and 2 reveals the full cluster structure, so the pair has a clearly stronger clustering tendency than either of its components alone.

Components 1 and 2 (Figure 2c). The points form two horizontal bands, at component 2 ≈ 0.25 and ≈ 0.75 , that extend evenly over the whole range of component 1. This is simply the bimodality of component 2 seen together with a uniform, independent component 1 ($r = 0.054$, mutual information at the independence level). The two lower clusters merge into one band, so only two groups are visible.

In summary, component 1 is uniformly distributed and independent of the other two components: it is a noise dimension that carries no cluster information. If it is included, every cluster

¹Coordinates are given as (component 0, component 2); the sizes and centres were obtained by splitting the plane at 0.5 in both components.

is stretched into a “rod” along the whole component-1 axis, so points of the same cluster can be far apart in the full space and the clusters become much less compact. Components 0 and 2, in contrast, depend on each other, and the clusters live in their joint subspace.

4 Entropies

For a subset of k dimensions, each dimension was divided into $\varphi = 64^{1/k}$ equal ranges ($\varphi = 4, 8$ and 64 for $k = 3, 2$ and 1), so that every grid has exactly $m = 64$ cells. A value x was assigned to range $[\varphi x]$ and the value $x = 1$ to the last range, so that the grid covers the closed cube $[0, 1]^k$. With p_i the fraction of the 1000 points in cell i , the probability-based entropy of equation (6.2) in [1] is

$$E = - \sum_{i=1}^m [p_i \ln p_i + (1 - p_i) \ln(1 - p_i)], \quad (1)$$

computed with the natural logarithm and $0 \ln 0 = 0$. Because m is the same for all subsets, all values share the same scale: $E = 0$ if all points fall into one cell, and the maximum $E_{\max} = \ln 64 - 63 \ln \frac{63}{64} \approx 5.151$ is reached when every cell holds the same fraction $p_i = 1/64$. A sample of 1000 uniformly distributed points gives $E = 5.119 \pm 0.006$ (mean $\pm$ standard deviation, Section 9), slightly below $E_{\max}$ because of random fluctuations. Table 1 lists the seven entropies.

Table 1: Probability-based entropies E (equation (1), $m = 64$ cells) of all combinations of data dimensions. Grid: ranges per dimension; empty cells: cells without any point; $E/E_{\max}$: entropy relative to its maximum $E_{\max} = 5.151$; rank 1 is the lowest entropy.

Dimensions	k	Grid	Empty cells	E	$E/E_{\max}$	Rank
$\{0, 1, 2\}$	3	$4 \times 4 \times 4$	9	4.794	93.1 %	4
$\{0, 1\}$	2	8×8	0	4.998	97.0 %	6
$\{0, 2\}$	2	8×8	10	4.482	87.0 %	1
$\{1, 2\}$	2	8×8	0	4.792	93.0 %	3
$\{0\}$	1	64	0	4.954	96.2 %	5
$\{1\}$	1	64	0	5.118	99.4 %	7
$\{2\}$	1	64	1	4.761	92.4 %	2

Range and variation. The seven values lie in the narrow interval 4.482–5.118, i.e. 87.0–99.4 % of $E_{\max}$. In absolute terms the differences look small, but they are large compared with the random variation of the entropy of uniform data (standard deviation 0.006): apart from $\{1, 2\}$ and $\{0, 1, 2\}$, which differ by only +0.002, any two values differ by more than five such standard deviations. Since a low entropy means that the points are concentrated in few cells instead of being spread evenly, the ranking

$$E_{\{0,2\}} < E_{\{2\}} < E_{\{1,2\}} \approx E_{\{0,1,2\}} < E_{\{0\}} < E_{\{0,1\}} < E_{\{1\}}$$

orders the combinations from the strongest to the weakest clustering tendency.

What the values indicate about the components.

- $\{0, 2\}$ has by far the lowest entropy (4.482), and 10 of its 64 cells are empty: the regions between the three clusters. This is the combination with the strongest clustering tendency.
- Component 1 has the highest entropy, $E_{\{1\}} = 5.118$, practically equal to that of uniform random data (5.119 ± 0.006). Moreover, adding component 1 to any subset increases the entropy: $E_{\{0,1\}} - E_{\{0\}} = +0.044$, $E_{\{1,2\}} - E_{\{2\}} = +0.031$ and $E_{\{0,1,2\}} - E_{\{0,2\}} = +0.312$. Because m is fixed, a new dimension brings no new cells but divides the existing resolution: a uniform, independent component spreads the points of every coarser cell evenly over several finer cells and therefore raises the entropy. Component 1 contributes nothing to the clustering tendency; it even masks the structure of the other components.
- Component 2 has the lowest single-dimension entropy (4.761), and adding it always decreases the entropy: $E_{\{0,2\}} - E_{\{0\}} = -0.472$, $E_{\{1,2\}} - E_{\{1\}} = -0.326$ and $E_{\{0,1,2\}} - E_{\{0,1\}} = -0.204$. It is the strongest individual contributor.
- Component 0 has an intermediate single entropy (4.954): clearly below the uniform level but well above component 2, so on its own it contributes moderately. Together with component 2 it contributes a lot, $E_{\{0,2\}} - E_{\{2\}} = -0.279$: the pair is more than the sum of its parts. In the three-dimensional grid, however, adding component 0 to $\{1, 2\}$ changes the entropy by only $+0.002$, a resolution effect discussed in Section 7.

5 Greedy backward selection

Backward selection starts from the full set of dimensions and at each stage removes the dimension whose removal gives the lowest entropy, as long as this lowers the entropy; following Aggarwal [1], the search stops when the entropy would increase (or not decrease significantly). Table 2 and Figure 3(a) show the stages.

Table 2: Stages of greedy backward selection. ΔE : change of the entropy relative to the current set; bold: lowest entropy of the stage.

Stage	Current set (E)	Candidate set	E	ΔE	Decision
1	$\{0, 1, 2\}$ (4.794)	$\{1, 2\}$ (remove 0)	4.792	-0.002	selected
		$\{0, 2\}$ (remove 1)	4.482	-0.312	
		$\{0, 1\}$ (remove 2)	4.998	+0.204	
2	$\{0, 2\}$ (4.482)	$\{2\}$ (remove 0)	4.761	+0.279	stop: E would increase
		$\{0\}$ (remove 2)	4.954	+0.472	

Stage 1. Starting from $\{0, 1, 2\}$ with $E = 4.794$, the three possible removals give $\{1, 2\}$ (4.792), $\{0, 2\}$ (4.482) and $\{0, 1\}$ (4.998). Removing component 1 gives by far the largest reduction (-0.312), so component 1 is dropped. Removing component 0 would lower the entropy only marginally (-0.002), and removing component 2 would increase it ($+0.204$).

Stage 2. From $\{0, 2\}$ ($E = 4.482$), removing component 0 gives $\{2\}$ with 4.761 and removing component 2 gives $\{0\}$ with 4.954. Both would increase the entropy (by $+0.279$ and $+0.472$), so the search stops.

The result is $\{0, 2\}$ with $E = 4.482$. The search evaluated six of the seven subsets; only $\{1\}$ was never computed.

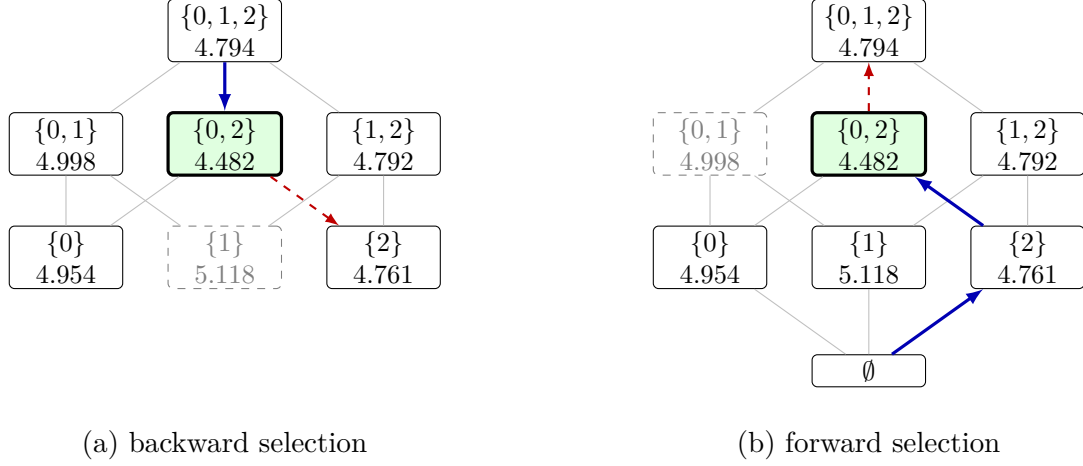


Figure 3: All subsets of dimensions with their entropies E and the paths of the two greedy searches. Solid blue arrows: accepted steps; dashed red arrow: best move of the last stage, rejected because E would increase; dashed grey box: subset never evaluated by the search; green box: result. Grey lines connect subsets that differ by one dimension.

6 Greedy forward selection

Forward selection starts from the empty set and at each stage adds the dimension that gives the lowest entropy, as long as this lowers the entropy. Table 3 and Figure 3(b) show the stages.

Table 3: Stages of greedy forward selection. ΔE : change of the entropy relative to the current set; bold: lowest entropy of the stage.

Stage	Current set (E)	Candidate set	E	ΔE	Decision
1	$\emptyset$	$\{0\}$ (add 0)	4.954	–	
		$\{1\}$ (add 1)	5.118	–	
		$\{2\}$ (add 2)	4.761	–	selected
2	$\{2\}$ (4.761)	$\{0, 2\}$ (add 0)	4.482	-0.279	selected
		$\{1, 2\}$ (add 1)	4.792	$+0.031$	
3	$\{0, 2\}$ (4.482)	$\{0, 1, 2\}$ (add 1)	4.794	$+0.312$	stop: E would increase

Stage 1. The empty set has no entropy on a 64-cell grid (all points would lie in a single cell), so the first stage simply takes the best single dimension: component 2 (4.761), ahead of component 0 (4.954) and component 1 (5.118).

Stage 2. Adding component 0 gives $\{0, 2\}$ with 4.482 (-0.279), adding component 1 gives $\{1, 2\}$ with 4.792 ($+0.031$). Component 0 is added.

Stage 3. The only remaining option, adding component 1, would increase the entropy to 4.794 ($+0.312$), so the search stops.

The result is $\{0, 2\}$ with $E = 4.482$, the same as in backward selection. Again six of the seven subsets were evaluated; this time $\{0, 1\}$ was never computed.

7 Analysis

A consistent picture. The visual inspection suggested that the clusters live in the plane of components 0 and 2 and that component 1 is uniform noise. The entropies confirm this: $\{0, 2\}$ has the lowest entropy, $\{1\}$ is at the level of uniform data, and every subset with component 1 has a higher entropy than the same subset without it. Both greedy searches end at $\{0, 2\}$, which is also the minimum of all seven values, i.e. the result an exhaustive search would give. This is what we expected from the plots. The decisive steps are also robust: the entropy changes of the accepted moves (0.28–0.31) are about fifty times the standard deviation of the entropy of uniform data, and the result stays the same with the Shannon entropy and with $m = 729$ (Section 9).

Why the two strategies agree here. The structure of these data is easy for greedy search. Component 1 is harmful in every combination, so backward selection removes it first, and components 0 and 2 are informative both individually and together, so forward selection builds $\{0, 2\}$ step by step. The two directions therefore meet at the same subset. In general this is not guaranteed: a greedy search never revisits a decision, and forward and backward selection can end at different subsets, neither of which needs to be optimal.

Where the greedy searches could fail. Forward selection judges each component first on its own. Our data contain a mild example of why this is risky: component 0 alone looks only weakly clustered (no gap in the histogram, $E_{\{0\}} = 4.954$), yet it is essential for separating the two lower clusters, which is visible only together with component 2. Forward selection found it because component 0 is not completely uniform and because in stage 2 it was evaluated together with component 2. If two components were uniform individually but clustered jointly (for example in a checkerboard pattern), forward selection would start from some other component and could miss the pair, whereas backward selection, which starts from the full space, can still detect the joint structure. Backward selection, on the other hand, makes its first decision on the coarsest grid, where the entropies are least informative, as the next paragraph shows.

The entropy depends on the resolution. Although the three clusters exist in the full space, the entropy of $\{0, 1, 2\}$ (4.794) is higher than those of $\{0, 2\}$ and $\{2\}$ and practically equal to that of $\{1, 2\}$; adding component 0 to $\{1, 2\}$ does not lower the entropy at all ($+0.002$). With m fixed, every added dimension lowers the resolution of all dimensions (64, 8 and 4 ranges for $k = 1, 2$ and 3). In the $4 \times 4 \times 4$ grid the range boundaries 0.25, 0.5 and 0.75 run right through the cluster centres (Section 3), so each cluster is split over 2×2 cells of the (0, 2)-plane, and the uniform component 1 divides each of these cells further into four

almost equally filled cells. The entropies therefore compare not only the data but also the grid resolutions, and comparisons across dimensionalities, which the stopping rules of both searches make, must be read with care. In our data this did not matter because removing component 1 was clearly the best first step, but with a less harmful noise component the coarse grid could have made backward selection drop component 0, a decision it could never undo.

Low entropy is not the same as clusters. The entropy measures how unevenly the points are spread over the cells, not whether they form separate groups. Component 0 has an entropy far below the uniform level ($z = -28.6$ in Section 9) mainly because its distribution is skewed, although its histogram shows no separated groups; a single dense blob would likewise have a low entropy. Visual inspection is needed to tell whether a low entropy comes from several separated groups or merely from a non-uniform density. In our data the low entropy of $\{0, 2\}$ does correspond to three real clusters, and that of $\{2\}$ to two groups.

8 Why 64 bins?

Why $m = 64$ was a good choice.

- $64 = 4^3 = 8^2 = 64^1$ is at the same time a perfect cube and a perfect square. Uniform grids with the same number of ranges in every dimension therefore give exactly 64 cells for every one-, two- and three-dimensional subset. In Aggarwal’s description the number of ranges per dimension is rounded, $\varphi = \lceil m^{1/k} \rceil$, so that m is only approximately the same for all subsets; with $m = 64$ no rounding is needed. Because the maximum entropy (and the entropy of uniform data) depends only on m , all seven values are on the same scale and can be compared directly. This is exactly what greedy search needs: its choices compare subsets of equal size, but its stopping decisions compare subsets of different dimensionality.
- It suits the amount of data: on average $1000/64 \approx 15.6$ points per cell, so the cell fractions p_i are estimated reasonably well (for uniform data the standard deviation of E is only 0.006), and practically no cell of uniform data would be empty. At the same time 64 ranges are fine enough to show the shapes of the one-dimensional distributions.
- $64 = 2^6$: the range widths $1/4$, $1/8$ and $1/64$ are powers of two, so the range index $\lfloor \varphi x \rfloor$ is computed without any rounding error in binary floating point, and the grids are nested (each 4-range boundary is also an 8-range and a 64-range boundary). For example, the 64-bin histograms can be merged exactly into the 8- or 4-range marginals of the coarser grids.

Other values of m . The numbers that are both squares and cubes are the sixth powers $m = q^6$: 1, 64, 729, 4096, ... $m = 1$ is useless, since every entropy would be 0. $m = 729 = 9^3 = 27^2$ is the only other candidate that does not exceed the number of points, but it gives only $1000/729 \approx 1.4$ points per cell: even for uniform data about a quarter of the cells would be empty, and the entropy of uniform data (7.168 ± 0.018) lies far below the maximum 7.591, so the entropy is dominated by sampling noise rather than by the structure of the data. Our extra experiment with $m = 729$ (Section 9) gives the same greedy result but much weaker contrasts. $m = 4096$ exceeds the number of points. With 1000 points, 64 is thus the only

sensible choice. If only subsets of one dimensionality were compared, any square (e.g. 16, 36, 100) would do for the pairs and any cube (e.g. 27, 125) for the triples; $64 = 2^6$ would even allow a six-dimensional grid with two ranges per dimension.

Higher-dimensional data. To compare all subsets of a d -dimensional data set with exactly the same m , m must be a perfect k th power for every $k \leq d$, i.e. $m = q^{\text{lcm}(1, \dots, d)}$: for $d = 4$ this means at least $2^{12} = 4096$ cells and for $d = 5$ at least $2^{60} \approx 10^{18}$ cells, which is impossible. Even without this requirement, a k -dimensional grid with at least two ranges per dimension has at least 2^k cells, so the number of points needed grows exponentially with k . Otherwise most cells are empty, almost every non-empty cell holds a single point, and E reflects the number of points rather than their structure (the curse of dimensionality). With higher-dimensional data one should therefore (i) choose m from the number of points (say, at least 5–10 points per cell on average), use $\varphi = \lceil m^{1/k} \rceil$ ranges per dimension and compare each entropy with its maximum or with the entropy of uniform data on the same grid, (ii) restrict the search to low-dimensional subsets, or (iii) use the entropy of the one-dimensional distribution of pairwise distances, which Aggarwal suggests as an alternative because its number of bins does not depend on the dimensionality.

9 Extra experiments

Table 4: Extra experiments on the seven combinations of dimensions: Shannon entropy H with $m = 64$ cells; standardised distance $z = (E - \mu)/\sigma$ of E (equation (1)) from the entropy of uniformly distributed data (mean μ , standard deviation σ over 10 000 simulated data sets); and E and z with $m = 729$ cells.

Dimensions	$m = 64$		$m = 729$	
	Shannon H	z of E	E	z of E
$\{0, 1, 2\}$	3.8064	−56.5	6.610	−30.8
$\{0, 1\}$	4.0084	−20.9	7.045	−6.8
$\{0, 2\}$	3.5031	−110.7	6.568	−33.1
$\{1, 2\}$	3.8060	−56.8	6.879	−15.9
$\{0\}$	3.9651	−28.6	7.020	−8.2
$\{1\}$	4.1264	−0.1	7.155	−0.7
$\{2\}$	3.7757	−62.2	6.867	−16.6

(i) Shannon entropy. Equation (1) adds the term $-\sum_i (1 - p_i) \ln(1 - p_i)$ to the Shannon entropy $H = -\sum_i p_i \ln p_i$. Table 4 shows H for all subsets. The difference $E - H$ is almost constant (between 0.979 and 0.992), because for small p_i the extra term is approximately $1 - \frac{1}{2} \sum_i p_i^2$. The ranking of the subsets is the same ($\{1, 2\}$ and $\{0, 1, 2\}$ differ only in the fourth decimal, 3.8060 and 3.8064), and both greedy searches again end at $\{0, 2\}$.

(ii) Baseline for uniform data. For uniformly distributed data each of the 64 equal cells has probability $1/64$ whatever the dimensionality, so the cell counts of 1000 points follow a

multinomial distribution. From 10 000 simulated data sets we obtained $E = 5.119 \pm 0.006$ (minimum 5.093). Component 1 is indistinguishable from uniform data ($z = -0.1$), whereas all other subsets lie far below this baseline, with z between -20.9 and -110.7 .

(iii) Pairwise dependence. Table 5 gives the Pearson correlation and the mutual information (MI, in nats) of each pair of components on the 8×8 grid. To judge the MI values, the second component of the pair was randomly permuted 2000 times, which destroys any dependence but keeps both marginal distributions. Only the pair $\{0, 2\}$ is dependent (MI 0.208 against 0.026 for permuted data); for $\{0, 1\}$ and $\{1, 2\}$ the MI is at the level expected for independent components. This confirms the visual impression of Section 3 and shows that the correlation coefficient alone ($r = 0.152$) would miss the nonlinear dependence between components 0 and 2.

Table 5: Dependence between pairs of components: Pearson correlation r , mutual information MI (nats) on the 8×8 grid, MI of 2000 permuted, i.e. independent, data sets, and the permutation p -value.

Pair	Pearson r	MI	MI of permuted data		
			mean	99 % quantile	p -value
$\{0, 1\}$	0.035	0.021	0.025	0.039	0.76
$\{0, 2\}$	0.152	0.208	0.026	0.039	< 0.001
$\{1, 2\}$	0.054	0.029	0.025	0.038	0.25

(iv) $m = 729$ cells. With $m = 729 = 9^3 = 27^2$ (9, 27 and 729 ranges per dimension) the maximum entropy is 7.591 and uniform data give $E = 7.168 \pm 0.018$. Both greedy searches again end at $\{0, 2\}$, and component 1 is again indistinguishable from uniform data ($z = -0.7$). The ranking changes, however: $\{0, 1, 2\}$ (6.610) moves from fourth to second place, just behind $\{0, 2\}$ (6.568), because nine ranges per dimension resolve the three clusters also in the full space, while the 27×27 and 729-range grids of the smaller subsets become sparse. The contrasts are much weaker (z down to -33.1 instead of -110.7) because the sampling noise grows with the number of cells. This again shows that grid entropies depend on the resolution and that m must fit the size of the data.

10 Conclusions

The data have a clear clustering tendency, but only in the subspace of components 0 and 2, where three well-separated clusters with about 50 %, 25 % and 25 % of the points appear. Component 1 is uniformly distributed and independent of the other two; it is a noise dimension without any clustering tendency that only blurs the clusters when it is included. Component 2 is the most informative single component (bimodal with a clear gap), whereas component 0 shows its value only together with component 2. The probability-based entropies agree with this picture: $\{0, 2\}$ has clearly the lowest entropy, $\{1\}$ has the entropy of uniform data, adding component 1 always increases the entropy, and both greedy backward and greedy forward selection end at the optimal subset $\{0, 2\}$.

The experiment also shows the limits of grid entropies. They measure how unevenly the points fill the cells, not whether they form separate groups, and because m is fixed, their resolution per dimension falls with the dimensionality, which is why the three-dimensional entropy understated the structure of the full space. Entropy-based measures of clustering tendency are therefore best used together with visual inspection, and for data with more dimensions the grid must be chosen with care; $m = 64$ worked well here because it is both a square and a cube and fits 1000 points.

References

- [1] C. C. Aggarwal. *Data Mining: The Textbook*. Springer, 2015. Chapter 6, equation (6.2).

11 Appendix – Code

The script `code/clustering_tendency.py` produces all figures, tables and numbers of this report (run `python3 code/clustering_tendency.py` in the project directory).

```

1  #!/usr/bin/env python3
2  """Clustering tendency and entropy-based measures (CS-E4650).
3
4  Reproduces every figure, table and number of the report:
5      1. loads the data and checks that it has 1000 three-dimensional vectors
6         with all values in [0, 1],
7      2. draws the 64-bin histograms and the pairwise scatter plots,
8      3. divides every subset of the dimensions into  $m = 64$  uniform grid cells
9         and computes the probability-based entropy of Aggarwal's eq. (6.2),
10     4. simulates greedy backward and forward selection with these entropies,
11     5. runs the extra experiments (Shannon entropy, uniform-data baseline,
12        pairwise dependence and  $m = 729$ ).
13
14  Usage: python3 code/clustering_tendency.py
15  """
16  import itertools
17  from pathlib import Path
18
19  import matplotlib
20
21  matplotlib.use("Agg") # write files only, no display needed
22  import matplotlib.pyplot as plt # noqa: E402
23  import numpy as np # noqa: E402
24
25  ROOT = Path(__file__).resolve().parents[1]
26  DATA_FILE = ROOT / "data" / "clustering-tendency.csv"
27  FIG_DIR = ROOT / "figures", ROOT / "results"
28  TEX_DIR = RES_DIR / "latex"
29
30  M = 64 # number of grid cells in every entropy calculation
31  M_EXTRA = 729 # the next m that is both a square and a cube (extra experiment)
32  DIMS = (0, 1, 2)
33  # all non-empty subsets of the dimensions: (0,1,2), (0,1), (0,2), (1,2), (0,), (1,), (2,)
34  SUBSETS = [s for k in (3, 2, 1) for s in itertools.combinations(DIMS, k)]
35  N_SIM = 10_000 # simulated uniform data sets for the entropy baseline
36  N_PERM = 2_000 # permutations in the independence test
37  rng = np.random.default_rng(2025)
38
39
40  # ----- data -----
41  def load_data(path=DATA_FILE):
42      """Load the data and check the properties stated in the task."""
43      X = np.loadtxt(path, delimiter=",")
44      assert X.shape == (1000, 3), f"unexpected shape {X.shape}"
45      assert np.isfinite(X).all(), "non-numeric values"
46      assert X.min() >= 0.0 and X.max() <= 1.0, "values outside [0, 1]"

```

```

47     return X
48
49
50 # ----- entropies ---
51 def ranges_per_dim(m, k):
52     """Ranges per dimension phi such that a k-dimensional grid has phi**k = m cells."""
53     phi = round(m ** (1.0 / k))
54     if phi**k != m:
55         raise ValueError(f"m = {m} is not the {k}th power of an integer")
56     return phi
57
58
59 def cell_probabilities(X, dims, m=M):
60     """Fractions p_1..p_m of the points in the m uniform grid cells of
61     [0,1]^k spanned by the dimensions `dims` (k = len(dims))."""
62     Y = X[:, list(dims)]
63     k = Y.shape[1]
64     phi = ranges_per_dim(m, k)
65     # range index floor(phi * x) of every value; x = 1 goes to the last range
66     idx = np.minimum(np.floor(Y * phi).astype(int), phi - 1)
67     counts = np.bincount(np.ravel_multi_index(idx.T, (phi,) * k), minlength=m)
68     # sanity check against numpy's own multidimensional histogram
69     edges = [np.linspace(0.0, 1.0, phi + 1)] * k
70     assert np.array_equal(counts, np.histogramdd(Y, bins=edges)[0].ravel())
71     return counts / len(Y)
72
73
74 def xlogx(p):
75     """Elementwise p ln p with the convention 0 ln 0 = 0."""
76     p = np.asarray(p, dtype=float)
77     out = np.zeros_like(p)
78     out[p > 0] = p[p > 0] * np.log(p[p > 0])
79     return out
80
81
82 def entropy(p):
83     """Probability-based entropy of Aggarwal's eq. (6.2), natural logarithm:
84     E = -sum_i [p_i ln p_i + (1 - p_i) ln(1 - p_i)]."""
85     return -np.sum(xlogx(p) + xlogx(1.0 - p), axis=-1)
86
87
88 def shannon(p):
89     """Shannon entropy -sum_i p_i ln p_i (extra experiment)."""
90     return -np.sum(xlogx(p), axis=-1)
91
92
93 def all_entropies(X, m=M, measure=entropy):
94     """Entropy of every non-empty subset of the dimensions."""
95     return {s: float(measure(cell_probabilities(X, s, m))) for s in SUBSETS}
96
97
98 # ----- greedy searches ---
99 def greedy_backward(E, dims=DIMS):
100     """Start from all dimensions and repeatedly drop the dimension whose
101     removal gives the lowest entropy; stop when even the best removal
102     would not decrease the entropy."""
103     current, stages = tuple(dims), []
104     while len(current) > 1:
105         candidates = [tuple(d for d in current if d != r) for r in current]
106         best = min(candidates, key=E.get)
107         accepted = E[best] < E[current]
108         stages.append((current, candidates, best, accepted))
109         if not accepted:
110             break
111         current = best
112     return current, stages
113
114
115 def greedy_forward(E, dims=DIMS):
116     """Start from the empty set and repeatedly add the dimension that gives
117     the lowest entropy; stop when even the best addition would not decrease
118     the entropy. The empty set has no entropy on a 64-cell grid, so the

```

```

119     best single dimension is always taken in the first stage."""
120     current, stages = (), []
121     while len(current) < len(dims):
122         candidates = [tuple(sorted(current + (a,))) for a in dims if a not in current]
123         best = min(candidates, key=E.get)
124         accepted = not current or E[best] < E[current]
125         stages.append((current, candidates, best, accepted))
126         if not accepted:
127             break
128         current = best
129     return current, stages
130
131
132 # ----- extra experiments ---
133 def uniform_baseline(m, n):
134     """Entropies of N_SIM simulated data sets of n uniformly distributed
135     points. Every cell of a uniform grid has probability 1/m whatever the
136     dimensionality, so the cell counts are multinomial(n; 1/m, ..., 1/m)."""
137     counts = rng.multinomial(n, np.full(m, 1.0 / m), size=N_SIM)
138     return entropy(counts / n)
139
140
141 def mutual_information(x, y, phi=8):
142     """Mutual information (nats) of two components on the phi x phi grid."""
143     p = cell_probabilities(np.column_stack([x, y]), (0, 1), phi * phi)
144     p = p.reshape(phi, phi)
145     return float(shannon(p.sum(axis=1)) + shannon(p.sum(axis=0)) - shannon(p.ravel()))
146
147
148 def dependence(X, i, j):
149     """Pearson correlation, mutual information and a permutation test of
150     independence (shuffling component j destroys any dependence)."""
151     mi = mutual_information(X[:, i], X[:, j])
152     null = np.array(
153         [mutual_information(X[:, i], rng.permutation(X[:, j])) for _ in range(N_PERM)]
154     )
155     return {
156         "r": float(np.corrcoef(X[:, i], X[:, j])[0, 1]),
157         "mi": mi,
158         "null_mean": float(null.mean()),
159         "null_q99": float(np.quantile(null, 0.99)),
160         "p": (1 + int(np.sum(null >= mi))) / (1 + N_PERM),
161     }
162
163
164 def describe_groups(X):
165     """Rough sizes and centres of the three groups seen in the (0, 2) plane,
166     separated by the lines x2 = 0.5 and x0 = 0.5 (quoted in the report)."""
167     low = X[:, 2] < 0.5
168     groups = {"A": low & (X[:, 0] < 0.5), "B": low & (X[:, 0] >= 0.5), "C": ~low}
169     return {g: (int(mask.sum()), X[mask].mean(axis=0)) for g, mask in groups.items()}
170
171
172 # ----- figures ---
173 plt.rcParams.update({"font.size": 9, "axes.titlesize": 10, "savefig.bbox": "tight"})
174
175
176 def plot_histograms(X):
177     """Histograms of the three components with M = 64 bins on [0, 1]."""
178     fig, axes = plt.subplots(3, 1, figsize=(7.0, 6.6), sharey=True)
179     for j, ax in enumerate(axes):
180         ax.hist(X[:, j], bins=M, range=(0, 1), color="tab:blue",
181                edgecolor="white", linewidth=0.4)
182         ax.axhline(len(X) / M, color="black", linestyle="--", linewidth=1,
183                  label=f"uniform expectation {len(X)}/{M} = {len(X) / M:.1f}")
184         ax.set(xlim=(0, 1), title=f"Component {j}",
185              xlabel=f"value of component {j}", ylabel="count")
186     axes[0].legend(loc="upper right", frameon=False)
187     fig.tight_layout()
188     fig.savefig(FIG_DIR / "histograms.pdf")
189     plt.close(fig)
190

```

```

191
192 def plot_scatter(X, i, j):
193     """Scatter plot of components i and j with equal axis scales. The grey
194     lines are the 8 x 8 grid used for the two-dimensional entropies."""
195     fig, ax = plt.subplots(figsize=(2.4, 2.4))
196     eighths = np.linspace(0, 1, 9)
197     ax.set_xticks([0, 0.5, 1])
198     ax.set_yticks([0, 0.5, 1])
199     ax.set_xticks(eighths, minor=True)
200     ax.set_yticks(eighths, minor=True)
201     ax.xaxis.set_major_formatter(lambda v, _: f"{v:g}")
202     ax.yaxis.set_major_formatter(lambda v, _: f"{v:g}")
203     ax.tick_params(labelsize=10)
204     ax.grid(which="both", color="0.85", linewidth=0.5)
205     ax.set_axisbelow(True)
206     ax.scatter(X[:, i], X[:, j], s=4, alpha=0.6, linewidths=0, color="tab:blue")
207     ax.set(xlim=(0, 1), ylim=(0, 1))
208     ax.set_xlabel(f"component {i}", fontsize=12)
209     ax.set_ylabel(f"component {j}", fontsize=12)
210     ax.set_aspect("equal")
211     fig.savefig(FIG_DIR / f"scatter_{i}_{j}.pdf")
212     plt.close(fig)
213
214
215 # ----- LaTeX output -----
216 def key(s):
217     return "".join(map(str, s))
218
219
220 def tex_set(s):
221     return r"${\text{" + ",".join(map(str, s)) + r"}}$" if s else r"${\text{emptyset}}$"
222
223
224 def entropy_table(E, empty, e_max):
225     rank = {s: r for r, s in enumerate(sorted(E, key=E.get), 1)}
226     rows = [r"\begin{tabular}{lcccccc}", r"\toprule",
227            r"Dimensions & $\text{" + ",".join(map(str, SUBSETS)) + r"}$ & Grid & Empty cells & $E$ & $E/E_{\text{" + ",".join(map(str, SUBSETS)) + r"}}$ & Rank \\\\"
228     for s in SUBSETS:
229         phi = ranges_per_dim(M, len(s))
230         if s in ((0, 1, 2), (0, 1), (0,)): # a rule before each dimensionality
231             rows.append(r"\midrule")
232         grid = "$" + r"\times".join([str(phi)] * len(s)) + "$"
233         rows.append(f"{tex_set(s)} & {len(s)} & {grid} & {empty[s]} & {E[s]:.3f} & "
234                    f"{100 * E[s] / e_max:.1f} \\\\", "\\\\" & {rank[s]} \\\\"")
235     rows += [r"\bottomrule", r"\end{tabular}"]
236     return "\n".join(rows) + "\n"
237
238
239 def greedy_table(E, stages, backward):
240     rows = [r"\begin{tabular}{c|lrrrl}", r"\toprule",
241            r"Stage & Current set ($E$) & Candidate set & $E$ & $\Delta E$ & Decision \\\\"
242     for n, (current, candidates, best, accepted) in enumerate(stages, 1):
243         rows.append(r"\midrule")
244         head = f"{n} & {tex_set(current)} & ({tex_set(candidates)}) & ({E[current]:.3f})" if current else ""
245         for c in candidates:
246             moved = (set(current) ^ set(c)).pop()
247             what = f"(remove {moved})" if backward else f"(add {moved})"
248             delta = f"${E[c]} - E[{current}]:+.3f}$" if current else "--"
249             value = f"${E[c]:.3f}$"
250             decision = ""
251             if c == best:
252                 value = r"\textbf{" + value + "}"
253                 decision = "selected" if accepted else r"stop: $E$ would increase"
254             rows.append(f"{head} & {tex_set(c)} & {what} & {value} & {delta} & {decision} \\\\"")
255         head = "&" # stage and current set only on the first row of a stage
256     rows += [r"\bottomrule", r"\end{tabular}"]
257     return "\n".join(rows) + "\n"
258
259
260 def extra_table(H, z, E_big, z_big):
261     rows = [r"\begin{tabular}{lcccccc}", r"\toprule",
262            r" & \multicolumn{2}{c}{c} & \multicolumn{2}{c}{c} & \multicolumn{2}{c}{c} \\\\"

```

```

263         r"\cmidrule(lr){2-3}\cmidrule(lr){4-5}",
264         r"Dimensions & Shannon  $H$  &  $Z$  of  $E$  &  $E_{big}$  &  $Z$  of  $E$  \\"]
265     for s in SUBSETS:
266         if s in ((0, 1, 2), (0, 1), (0,)):
267             rows.append(r"\midrule")
268             rows.append(f"{tex_set(s)} & {H[s]:.4f} & {Z[s]:.1f} & {E_big[s]:.3f} & "
269                         f"{Z_big[s]:.1f} \\")
270     rows += [r"\bottomrule", r"\end{tabular}"]
271     return "\n".join(rows) + "\n"
272
273
274 def dependence_table(dep):
275     rows = [r"\begin{tabular}{lcccc}", r"\toprule",
276             r" & & \multicolumn{2}{c}{MI of permuted data} & \\",
277             r"\cmidrule(lr){4-5}",
278             r"Pair & Pearson  $r$  & MI & mean & 99\% quantile &  $p$ -value \\",
279             r"\midrule"]
280     for (i, j), d in dep.items():
281         p = r"$<0.001$ if  $d[p]$  < 0.001 else  $f[d[p]:.2f]$ "
282         rows.append(f"{tex_set((i, j))} & {d['r']:.3f} & {d['mi']:.3f} & "
283                     f"{d['null_mean']:.3f} & {d['null_q99']:.3f} & {p} \\")
284     rows += [r"\bottomrule", r"\end{tabular}"]
285     return "\n".join(rows) + "\n"
286
287
288 def write_macros(values):
289     """Numbers quoted in the report text, read by report.tex as \val{name}."""
290     with open(TEX_DIR / "macros.tex", "w") as f:
291         f.write("% generated by code/clustering_tendency.py -- do not edit\n")
292         for name, value in values.items():
293             f.write(f"\expandafter\def\csname val@{name}\endcsname{{{value}}}\n")
294
295
296 # ----- main -----
297 def main():
298     for d in (FIG_DIR, RES_DIR, TEX_DIR):
299         d.mkdir(parents=True, exist_ok=True)
300     X = load_data()
301     n = len(X)
302
303     # visual inspection (tasks 2-3)
304     plot_histograms(X)
305     for i, j in itertools.combinations(DIMS, 2):
306         plot_scatter(X, i, j)
307
308     # entropies with m = 64 cells (tasks 5-7)
309     P = {s: cell_probabilities(X, s) for s in SUBSETS}
310     E = {s: float(entropy(p)) for s, p in P.items()}
311     empty = {s: int(np.sum(p == 0)) for s, p in P.items()}
312     e_max = float(entropy(np.full(M, 1.0 / M)))
313
314     # greedy searches (tasks 9-10)
315     back, back_stages = greedy_backward(E)
316     fwd, fwd_stages = greedy_forward(E)
317
318     # extra experiments
319     H = all_entropies(X, M, shannon)
320     base = uniform_baseline(M, n)
321     z = {s: (E[s] - base.mean()) / base.std() for s in SUBSETS}
322     E_big = all_entropies(X, M_EXTRA)
323     base_big = uniform_baseline(M_EXTRA, n)
324     z_big = {s: (E_big[s] - base_big.mean()) / base_big.std() for s in SUBSETS}
325     e_max_big = float(entropy(np.full(M_EXTRA, 1.0 / M_EXTRA)))
326     dep = {(i, j): dependence(X, i, j) for i, j in itertools.combinations(DIMS, 2)}
327     groups = describe_groups(X)
328     greedy_extra = {
329         "Shannon, m=64": (greedy_backward(H)[0], greedy_forward(H)[0]),
330         "eq. (6.2), m=729": (greedy_backward(E_big)[0], greedy_forward(E_big)[0]),
331     }
332
333     # --- text summary
334     out = [f"data: {X.shape[0]} vectors x {X.shape[1]} components, "]

```

```

335         f"min {X.min():.4f}, max {X.max():.4f}, "
336         f"values exactly 0 or 1: {int(np.sum((X == 0) | (X == 1)))}",
337         f"max entropy (uniform p_i = 1/{M}): E_max = {e_max:.4f}",
338         f"uniform data, n = {n}: E = {base.mean():.4f} +- {base.std():.4f}",
339         "", "subset      grid      empty E(6.2)   E/E_max Shannon z(unif) E(m=729) z(m=729)"
340     for s in SUBSETS:
341         phi = ranges_per_dim(M, len(s))
342         out.append(f"{str(s):10s} {phi:>3d}^{len(s)} {empty[s]:>5d} {E[s]:.4f} "
343                   f"{E[s] / e_max:6.1f} {H[s]:.4f} {z[s]:7.1f} {E_big[s]:.4f} "
344                   f"{z_big[s]:8.1f}")
345     for title, stages, result in (("backward", back_stages, back),
346                                   ("forward", fwd_stages, fwd)):
347         out += ["", f"greedy {title} selection:"]
348         for current, candidates, best, accepted in stages:
349             cands = ", ".join(f"{c}: {E[c]:.4f}" for c in candidates)
350             cur = f"{current} (E = {E[current]:.4f})" if current else "()"
351             out.append(f"    from {cur}: {cands} -> "
352                       f"{'take ' + str(best) if accepted else 'stop'}")
353             out.append(f"    result: {result}, E = {E[result]:.4f}")
354     out += ["", f"uniform data, m = {M_EXTRA}: E_max = {e_max_big:.4f}, "
355           f"E = {base_big.mean():.4f} +- {base_big.std():.4f}"]
356     for name, (b, f) in greedy_extra.items():
357         out.append(f"greedy with {name}: backward {b}, forward {f}")
358     out.append("")
359     for (i, j), d in dep.items():
360         out.append(f"pair ({i},{j}): r = {d['r']:.3f}, MI = {d['mi']:.4f}, "
361                   f"MI under independence {d['null_mean']:.4f} "
362                   f"(99% {d['null_q99']:.4f}), p = {d['p']:.4f}")
363     for g, (size, centre) in groups.items():
364         out.append(f"group {g}: {size} points, mean {np.round(centre, 3)}")
365     summary = "\n".join(out)
366     print(summary)
367     (RES_DIR / "summary.txt").write_text(summary + "\n")
368
369     # ---- machine-readable results
370     with open(RES_DIR / "entropies.csv", "w") as f:
371         f.write("dimensions,k,ranges_per_dim,cells,empty_cells,E_eq6_2,E_over_Emax,"
372               "shannon,z_uniform,E_eq6_2_m729,z_uniform_m729\n")
373     for s in SUBSETS:
374         f.write(f"{key(s)}, {len(s)}, {ranges_per_dim(M, len(s))}, {M}, {empty[s]}, "
375               f"{E[s]:.6f}, {E[s] / e_max:.6f}, {H[s]:.6f}, {z[s]:.3f}, "
376               f"{E_big[s]:.6f}, {z_big[s]:.3f}\n")
377
378     # ---- LaTeX tables and macros for the report
379     (TEX_DIR / "entropy_table.tex").write_text(entropy_table(E, empty, e_max))
380     (TEX_DIR / "backward_table.tex").write_text(greedy_table(E, back_stages, True))
381     (TEX_DIR / "forward_table.tex").write_text(greedy_table(E, fwd_stages, False))
382     (TEX_DIR / "extra_table.tex").write_text(extra_table(H, z, E_big, z_big))
383     (TEX_DIR / "dependence_table.tex").write_text(dependence_table(dep))
384
385     counts1 = np.round(P[(1,)] * n).astype(int)
386     v = {"N": n, "NBoundary": int(np.sum((X == 0) | (X == 1))),
387          "E_max": f"{e_max:.3f}", "E_maxBig": f"{e_max_big:.3f}",
388          "UnifMean": f"{base.mean():.3f}", "UnifSd": f"{base.std():.3f}",
389          "UnifMin": f"{base.min():.3f}",
390          "UnifMeanBig": f"{base_big.mean():.3f}", "UnifSdBig": f"{base_big.std():.3f}",
391          "HistMinOne": counts1.min(), "HistMaxOne": counts1.max(),
392          "LowTwo": int(np.sum(X[:, 2] < 0.5)), "HighTwo": int(np.sum(X[:, 2] >= 0.5)),
393          "GapTwo": int(np.sum((X[:, 2] >= 0.45) & (X[:, 2] < 0.55))),
394          "NSim": f"{N_SIM:,}".replace(",", " r\\,"), "NPerm": f"{N_PERM:,}".replace(",", " r\\,"),
395          "Back": tex_set(back), "Fwd": tex_set(fwd)}
396     for s in SUBSETS:
397         v |= {"E{key(s)}": f"{E[s]:.3f}", "H{key(s)}": f"{H[s]:.4f}",
398              "Z{key(s)}": f"{z[s]:.1f}", "EBig{key(s)}": f"{E_big[s]:.3f}",
399              "ZBig{key(s)}": f"{z_big[s]:.1f}", "Empty{key(s)}": empty[s],
400              "Rel{key(s)}": f"{100 * E[s] / e_max:.1f}",
401              "Diff{key(s)}": f"{E[s] - H[s]:.3f}"}
402     for t in SUBSETS:
403         v[f"D{key(s)}-{key(t)}"] = f"{E[s] - E[t]:+.3f}"
404     for (i, j), d in dep.items():
405         v |= {"R{i}{j}": f"{d['r']:.3f}", "MI{i}{j}": f"{d['mi']:.3f}",
406              "MINull{i}{j}": f"{d['null_mean']:.3f}"}

```

```
407     for g, (size, centre) in groups.items():
408         v |= {f"Size{g}": size, f"Centre{g}": f"({centre[0]:.2f}, {centre[2]:.2f})"}
409     for name, (b, f) in zip(("Sh", "Big"), greedy_extra.values()):
410         v |= {f"Back{name}": tex_set(b), f"Fwd{name}": tex_set(f)}
411     write_macros(v)
412
413
414 if __name__ == "__main__":
415     main()
```
